

Task 1: Live Product Gallery

Objective: Create a dynamic product catalog that fetches real-time data from an API and displays it using state management.

1. Data Fetching & State

You will manage the data directly within your main component using **React Hooks**:

- **useState:** Create a state variable to hold your list of products (initialize it as an empty array []).
- **useEffect:** Use this hook to fetch data from a public API (e.g., <https://fakestoreapi.com/products>, <https://api.escuelajs.co/api/v1/products>, <https://dummyjson.com/products>,) as soon as the page loads.
- **Fetching Logic:** Use `fetch()` with `.then()` or `async/await` to get the data and update your state.

2. Displaying the Data (List Rendering)

Instead of a separate card component, render the UI directly inside your `.map()` function:

- Map through your state array to generate a "Card" layout for each item.
- **Required Fields:** Display the product **image**, **title**, **price**, and **category**.

3. Loading & Error States

Because backend data isn't instant, you must handle the "waiting" period:

- Show a simple "Loading products..." message or a Spinner while the data is being fetched.
- If the API fails, display a message like "Failed to load products. Please try again later."

4. UI Library Integration (Styling)

Use your chosen UI library (Material UI, DaisyUI, etc.) to make the list look professional:

- **Grid Layout:** Use a Responsive Grid so the products align properly (e.g., 1 column on phone, 3 on desktop).
- **Status Badge:** Since many APIs don't provide a "Stock Status," create a logic gate in your code:
 - If the price is over \$100 → Show a "**Premium**" badge (Blue).
 - If the price is under \$100 → Show a "**Best Seller**" badge (Green).

5. Search Bar

Add an `<input>` field at the top of the page. Use a `.filter()` method on your data so that the list updates in real-time as the user types a product name.